

LAB 6

MỤC TIÊU

Kết thúc bài thực hành sinh viên có khả năng:

- ✓ Đồng bộ hóa HUD để hiển thị thông tin như thanh máu và điểm số giữa các client.
- ✓ Xử lý sự kiện va chạm vật lý trong môi trường mạng.

NỘI DUNG

BÀI 1:

Sinh viên cần thực hiện:

- Tạo thanh máu (Health Bar) trong giao diện HUD.
- Đồng bộ dữ liệu máu giữa các client bằng Networked Properties.
- Hiển thị thanh máu được cập nhật theo thời gian thực.

BÀI 2:

Sinh viên cần thực hiện:

- Hiển thị điểm số trên HUD.
- Cập nhật điểm số khi tiêu diệt đối thủ.

BÀI 3:

Sinh viên cần thực hiện:

- Thêm logic phát hiện va chạm vật lý.
- Đồng bộ sự kiện va chạm giữa các client.
- Xử lý giảm máu khi nhân vật bị tấn công.

BÀI 4: GV CHO THÊM

***** YÊU CẦU NỘP BÀI:**

Sv nén file bao gồm các yêu cầu đã thực hiện trên, nộp lms đúng thời gian quy định của giảng viên. Không nộp bài coi như không có điểm.

--- Hết ---

HƯỚNG DẪN

BÀI 1:

Bước 1: Tạo thanh máu trong HUD

- ✓ Tạo UI thanh máu:
 - Tạo một Canvas trong Scene, thêm Slider UI.
 - Đặt tên Slider là HealthBar, chỉnh giá trị Max là 100
- ✓ Tạo script HUDController và gắn vào Canvas:

```
public class HUDController : MonoBehaviour
{
    public Slider healthBar;
    private PlayerStats playerStats;
    private void Start()
    {
        playerStats = FindObjectOfType<PlayerStats>();
    }
    private void Update()
    {
        if (playerStats != null)
        {
            healthBar.value = playerStats.HP;
        }
    }
}
```

Bước 2: Đồng bộ hóa thanh máu giữa các client

- ✓ Thêm thuộc tính HP trong PlayerStats.

```
public class PlayerStats : NetworkBehaviour
{
    [Networked] public int HP { get; set; }
    private void Start()
    {
        if (Object.HasInputAuthority)
        {
            HP = 100; // Thiết lập HP mặc định
        }
    }
    public void TakeDamage(int damage)
    {
        if (Object.HasStateAuthority)
        {
            HP = Mathf.Max(HP - damage, 0);
        }
    }
}
```

✓ Cập nhật logic giảm máu

```
private void Update()
{
    if (Object.HasInputAuthority && Input.GetKeyDown(KeyCode.H))
    {
        RPC_TakeDamage(10);
    }
}

[Rpc(RpcSources.InputAuthority, RpcTargets.StateAuthority)]
private void RPC_TakeDamage(int damage)
{
    TakeDamage(damage);
}
```

BÀI 2:

Bước 1: Hiển thị điểm số trên HUD

✓ Cập nhật Canvas:

Thêm một Text UI vào Canvas để hiển thị điểm số.

Đặt tên là ScoreText.

✓ Cập nhật script HUDController:

```
public TextMeshProUGUI scoreText;
private void Update()
{
    if (playerStats != null)
    {
        healthBar.value = playerStats.HP;
        scoreText.text = $"Score: {playerStats.Score}";
    }
}
```

Bước 2: Đồng bộ hóa điểm số

- ✓ Thêm thuộc tính Score trong PlayerStats.

```
[Networked] public int Score { get; set; }
public void AddScore(int points)
{
    if (Object.HasStateAuthority)
    {
        Score += points;
    }
}
```

- ✓ Cập nhật logic tăng điểm: Khi nhấn P, tăng điểm

```
private void Update()
{
    if (Object.HasInputAuthority && Input.GetKeyDown(KeyCode.P))
    {
        RPC_AddScore(5);
    }
}

[Rpc(RpcSources.InputAuthority, RpcTargets.StateAuthority)]
private void RPC_AddScore(int points)
{
    AddScore(points);
}
```

Bước 3: Chạy game và kiểm tra

- ✓ Chạy trò chơi, nhấn P để tăng điểm.
- ✓ Kiểm tra điểm số cập nhật trên cả hai client.

BÀI 3:**Gợi ý****Thêm logic phát hiện va chạm:**

- ✓ Thêm Collider cho nhân vật.
- ✓ Cập nhật script PlayerStats.

Đồng bộ sự kiện va chạm:

- ✓ Bắt sự kiện va chạm của nhân vật
- ✓ Điều chỉnh logic va chạm